

A quick look at *index.html* shows the frequency of the source code statements' execution. This can suggest test or source code enhancements:

```
1      : #include <stdio.h>
2      : #include <stdlib.h>
3      : int compare_function(int a, int b)
4      4 : {
5      4 :     int result = 0;
6      :     int 'p;
7      4 :     if ( a > b ) {
8      1 :         result = 1;
9      3 :     } else if ( a < b ){
10     2 :         result = -1;
11     :     }
12     :
13     :     //p = malloc(sizeof(int) * 10);
14     :     //free(p);
15     4 :     return result;
16     : }
```

A closer look at coverage can unearth redundant code, unexpected branches then, functions that are not executed, potential bugs, and so on.

Let's take a separate code coverage example to illustrate a simple case:

```
1      1: int var = 4;
2      1: if(var = 5) {
3      1:     printf(" 5 ");
4      1: }
```

In the example above, the *if* statement on Line 2 was intended to compare the value of the variable *var* with the numeric value 5. Due to a typo, it instead accidentally assigns the value of 5 to the *var* variable. The coverage shows that the branch is taken at line Number 2, which is an unexpected path. On investigating the reason, it becomes obvious that a typo has occurred, and it can then be corrected.



Tip: In your code, suppose a function *a()* calls *b()*, and *b()* in turn calls *c()*, always try to call the function *a()* in your unit testing, providing it with the

necessary data. This adds practical value to unit testing and coverage analysis.

Memory leak checking

Valgrind is a powerful binary-level debugging and profiling tool for executables, which is employed to check for memory leaks, cache profiling, deadlock detection and so on. You need to compile your code with the *-g* option to generate more debugging information in your executables.

Memory leaks and similar errors occur when *free()* or *delete* is used inappropriately on allocated data, if a double *free* is done, or allocated data is not freed.

The generic *valgrind* invocation syntax is as follows:

```
bash> valgrind --tool=toolname <program arguments>
```

To see *valgrind* in action, uncomment lines 13 and 14 in *test.c*, which allocate and free memory. Rebuild the test binary (run *make* again). Proceed to run *test_ut.bin* again, as shown below, to view a sample of memory leakage detection:

```
bash> make unittest_cov
bash> valgrind ./test_ut.bin
==14817== Memcheck, a memory error detector
==14817== Copyright (C) 2002-2009, and GNU GPL'd, by Julian Seward et al.
==14817== Using Valgrind-3.5.0 and LibVEX; rerun with -h for copyright info
==14817== Command: ./test_ut.bin
==14817==
Success: 4 tests passed.
Test time: 0.07 seconds.
==14817==
==14817== HEAP SUMMARY:
==14817==    in use at exit: 160 bytes in 4 blocks
==14817== total heap usage: 5 allocs, 1 frees, 512 bytes allocated
==14817==
==14817== LEAK SUMMARY:
==14817==    definitely lost: 160 bytes in 4 blocks
==14817==    indirectly lost: 0 bytes in 0 blocks
==14817==    possibly lost: 0 bytes in 0 blocks
==14817==    still reachable: 0 bytes in 0 blocks
==14817==    suppressed: 0 bytes in 0 blocks
==14817== Rerun with --leak-check=full to see details of leaked memory
==14817==
```

Asia's Leading Open Source Magazine

To BUY or SUBSCRIBE, please visit www.linuxforu.com

www.linuxforu.com

